

中山大学计算机学院**人工智能****本科生实验报告****(2022 学年春季学期)**

课程名称: Artificial Intelligence

教学班级	人工智能（饶洋辉 老师）	专业（方 向）	23 级地理信息科学（辅 修）
学号	23306103	姓名	赵义凯

一、 实验题目

使用 PyTorch 框架实现 MLP 模型并应用于房价预测与 Mnist 分类任务（选择实现两个任务）

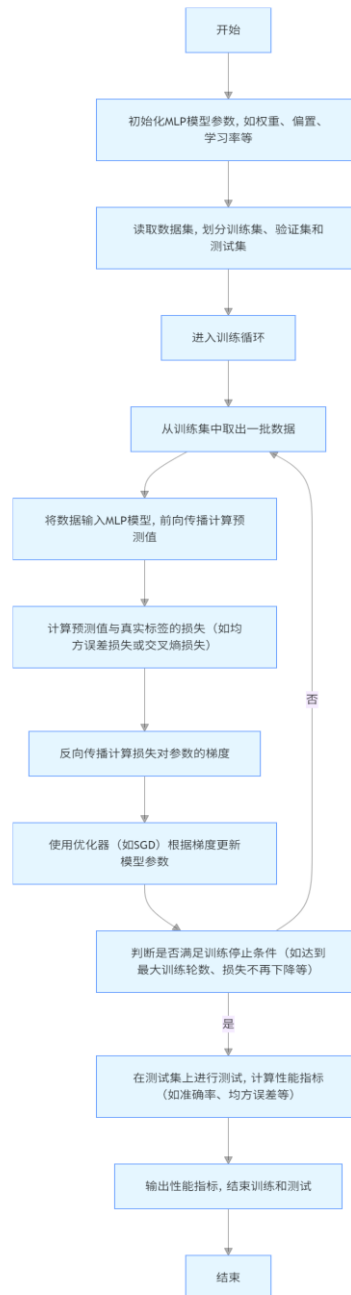
二、 实验内容**1. 算法原理**

多层感知机（MLP）是一种前馈神经网络，由输入层、多个隐藏层和输出层组成。每个神经元与下一层的所有神经元相连，通过权重和偏置传递信息。在训练过程中，利用反向传播算法计算损失函数对权重和偏置的梯度，并使用优化器（如随机梯度下降 SGD）更新参数，以最小化损失函数。

在房价预测任务中，输入层接收经度、纬度、房龄、房主收入等特征，隐藏层对这些特征进行非线性变换，输出层预测房价。

在 Mnist 分类任务中，输入层接收 28×28 的灰度图像数据，经过隐藏层处理后，输出层预测图像对应的数字标签。

2. 伪代码**2.1 流程图**



2.2 伪代码（含回归和分类任务）



```
# 初始化神经网络模型
model = initialize_model()
# 定义损失函数
loss_function = define_loss_function()
# 定义优化器
optimizer = define_optimizer(model.parameters())
# 定义训练轮数
epochs = define_number_of_epochs()
# 加载数据集
train_dataset, test_dataset = load_datasets()
# 创建数据加载器
train_loader = create_data_loader(train_dataset)
test_loader = create_data_loader(test_dataset)
# 训练循环
for epoch in range(epochs):
    # 设置模型为训练模式
    model.train()
    # 初始化训练损失
    train_loss = 0

    # 遍历训练数据加载器
    for batch_inputs, batch_labels in train_loader:
        # 梯度清零
        optimizer.zero_grad()

        # 前向传播
        predictions = model(batch_inputs)

        # 计算损失
        loss = loss_function(predictions, batch_labels)

        # 反向传播
        loss.backward()

        # 更新模型参数
        optimizer.step()

        # 累加训练损失
        train_loss = train_loss + loss.item()

    # 计算平均训练损失
    average_train_loss = train_loss / len(train_loader)

    # 打印训练信息
    print(f'Epoch {epoch + 1}/{epochs}, Train Loss: {average_train_loss}')

    # 设置模型为评估模式
    model.eval()
    # 初始化测试损失和正确预测数
    test_loss = 0
    correct_predictions = 0

    # 关闭梯度计算
    with torch.no_grad():
        # 遍历测试数据加载器
        for batch_inputs, batch_labels in test_loader:
            # 前向传播
            predictions = model(batch_inputs)

            # 计算损失
            loss = loss_function(predictions, batch_labels)

            # 累加测试损失
            test_loss = test_loss + loss.item()

        # 计算正确预测数
        if task == "classification":
            predicted_classes = get_predicted_classes(predictions)
            correct_predictions = correct_predictions + count_correct_predictions(predicted_classes, batch_labels)

    # 计算平均测试损失
    average_test_loss = test_loss / len(test_loader)

    if task == "classification":
        # 计算准确率
        accuracy = correct_predictions / len(test_dataset)
        # 打印测试信息
        print(f'Test Loss: {average_test_loss}, Test Accuracy: {accuracy}')
    else:
        # 打印测试信息
        print(f'Test Loss: {average_test_loss}')
```



3. 关键代码展示（带注释）

3.1 房价预测代码（关键部分）

```
# 自定义房价数据集类
class HousePriceDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.float32)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

# 定义房价预测回归模型
class HousePricePredictor(nn.Module):
    def __init__(self):
        super(HousePricePredictor, self).__init__()
        self.fc1 = nn.Linear(4, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x

# 加载数据
def load_data(file_path):
    data = pd.read_csv(file_path, header=0)
    features = data.iloc[:, :4].values
    labels = data.iloc[:, 4].values
    return features, labels

# 处理异常值
def remove_outliers(features, labels):
    z_scores = np.abs((features - np.mean(features, axis=0)) / np.std(features, axis=0))
    mask = np.all(z_scores < 3, axis=1)
    return features[mask], labels[mask]

# 数据预处理
def preprocess_data(features, labels):
    features, labels = remove_outliers(features, labels)
    # 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2, random_state=42)
    return X_train, X_test, y_train, y_test

# 训练模型
def train_model(model, train_loader, criterion, optimizer, epochs):
    model.train()
    for epoch in range(epochs):
        running_loss = 0.0
        for batch_idx, (inputs, labels) in enumerate(train_loader):
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs.squeeze(), labels)
            loss.backward()
            # 添加梯度裁剪
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            running_loss += loss.item()
        print(f'Epoch {epoch + 1}/{epochs}, Loss: {running_loss / len(train_loader)}')

# 评估模型
def evaluate_model(model, test_loader, criterion):
    model.eval()
    total_loss = 0
    with torch.no_grad():
        for batch_idx, (inputs, labels) in enumerate(test_loader):
            outputs = model(inputs)
            loss = criterion(outputs.squeeze(), labels)
            total_loss += loss.item()
    print(f'Test Loss: {total_loss / len(test_loader)}')
```



3.2 Minist 数据集分类任务代码（关键部分）

```
# 定义MLP模型
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, output_size)
        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# 超参数设置
input_size = 28 * 28 # 每张图片的像素数 (28x28)
hidden_size = 128 # 隐藏层大小
output_size = 10 # 输出为10类数字 (0-9)
batch_size = 128
epochs = 10
learning_rate = 0.001

# 数据预处理
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5), (0.5)) # 归一化
])

# 数据集根目录
root_dir = r"D:\大二下\人工智能资料\实验课\lab_7_pytorch和MLP\mnist.jpg"
# 加载训练数据集
train_dataset = MNISTDataset(root_dir=root_dir, dataset_type='training', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
# 加载测试数据集
test_dataset = MNISTDataset(root_dir=root_dir, dataset_type='test', transform=transform)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
# 直接使用GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 初始化模型并移动到GPU
model = MLP(input_size=input_size, hidden_size=hidden_size, output_size=output_size).to(device)
# 损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
# 学习率调度器
scheduler = StepLR(optimizer, step_size=3, gamma=0.1)
# 用于记录每个epoch的损失和准确率
train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []

# 训练模型
if __name__ == "__main__":
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0
        for inputs, labels in train_loader:
            inputs = inputs.view(-1, 28 * 28).to(device) # 扁平化28x28的图片为 一维向量并移动到GPU
            labels = labels.to(device)
            # 前向传播
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            # 反向传播
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            # 统计损失和准确率
            running_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
        # 计算每个epoch的训练损失和准确率
        epoch_train_loss = running_loss / len(train_loader)
        train_accuracy = 100 * correct / total
        train_losses.append(epoch_train_loss)
        train_accuracies.append(train_accuracy)
        # 测试模型
        model.eval()
        running_test_loss = 0.0
        test_correct = 0
        test_total = 0
        with torch.no_grad():
            for test_inputs, test_labels in test_loader:
                test_inputs = test_inputs.view(-1, 28 * 28).to(device)
                test_labels = test_labels.to(device)
                test_outputs = model(test_inputs)
                test_loss = criterion(test_outputs, test_labels)
                running_test_loss += test_loss.item()
                _, test_predicted = torch.max(test_outputs, 1)
                test_total += test_labels.size(0)
                test_correct += (test_predicted == test_labels).sum().item()
            # 计算每个epoch的测试损失和准确率
            epoch_test_loss = running_test_loss / len(test_loader)
            test_accuracy = 100 * test_correct / test_total
            test_losses.append(epoch_test_loss)
            test_accuracies.append(test_accuracy)
        # 更新学习率
        scheduler.step()
        # 打印每个epoch的损失和准确率
        print(f'Epoch [{epoch + 1}/{epochs}], '
              f'Train Loss: {epoch_train_loss:.4f}, Train Accuracy: {train_accuracy:.2f}%, '
              f'Test Loss: {epoch_test_loss:.4f}, Test Accuracy: {test_accuracy:.2f}%')
```

4. 创新点&优化（如果有）

- 1) 可视化训练：使用 matplotlib 库的 **pyplot** 函数实现训练过程的可视化，将每一轮次训练的损失都用折线图的方式展现出来，并且在房价预测任务中还绘制了预测点和真实点的折线图，更加直观
- 2) 在 Mnist 数据集分类任务中采用学习率调度器 **Steplr**，帮助训练过程，同时也采用的 **adam** 优化器进行优化，提升训练效果，同时引入 softmax 层适用于多分类任务

三、 实验结果及分析

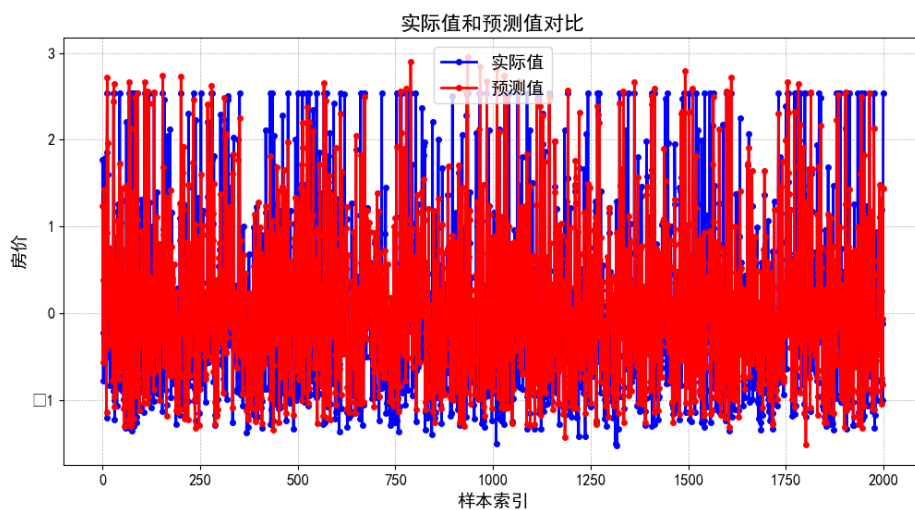
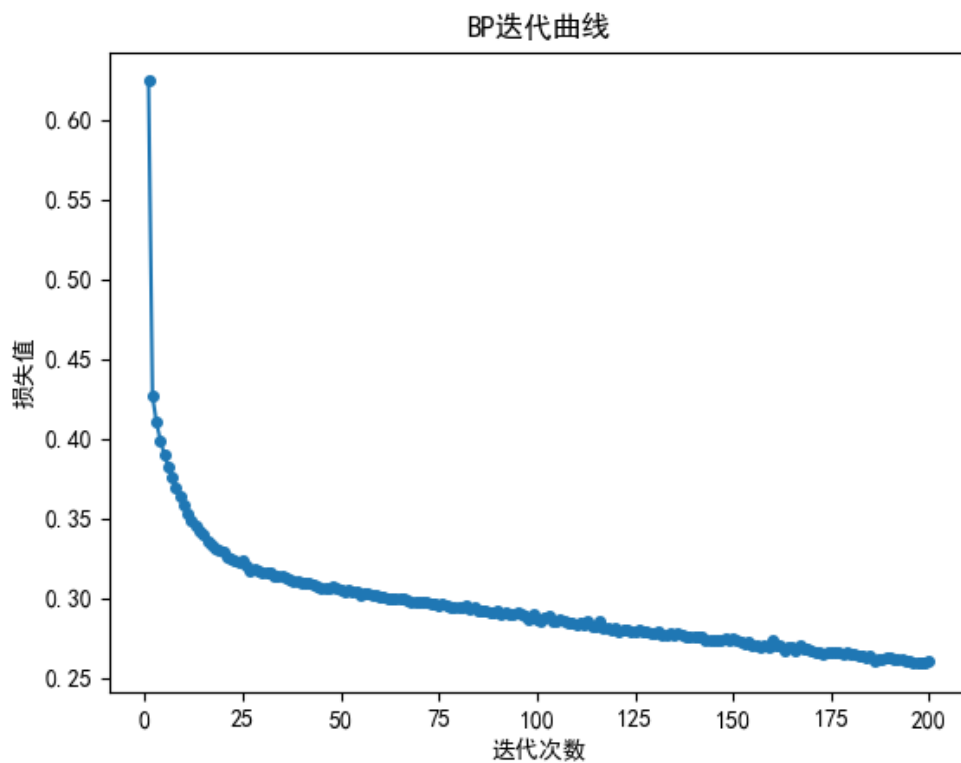
1. 房价预测任务

超参数：

- test_size = 0.2
- random_state = 42
- 输入层神经元数量：4，
- 隐藏层 1 神经元数量：100
- 隐藏层 2 神经元数量：50
- 输出层神经元数量：1
- batch_size = 64
- epochs = 500
- lr = 0.001:
- momentum = 0.9:
- max_norm = 1.0
- nn.MSELoss()
- optim.SGD

结果展示：数据集按照 0.2 和 0.8 分为验证集和训练集，使用 MSE 和 R^2 作为评价标准。首先将训练轮次设置为 200，得到损失（MSE）为 0.28， R^2 为 0.72，训练日志、反向传播损失曲线、训练集上真实值与预测值折线图如下所示。

```
Epoch 194/200, Loss: 0.2600749380588531
Epoch 195/200, Loss: 0.2607418057322502
Epoch 196/200, Loss: 0.2590259963274002
Epoch 197/200, Loss: 0.2591001546382904
Epoch 198/200, Loss: 0.25916046833992007
Epoch 199/200, Loss: 0.2594646618962288
Epoch 200/200, Loss: 0.26037955230474474
Test Loss (MSE): 0.2832539975643158
R^2 Score: 0.7239200469226028
D:\anaconda3\envs\unet\lib\tkinter\__init__.py:814: UserWarning: Glyph 8722 (\N{MINUS SIGN}) missing from current font.
  func(*args)
D:\anaconda3\envs\unet\lib\tkinter\__init__.py:1892: UserWarning: Glyph 8722 (\N{MINUS SIGN}) missing from current font.
  return self.func(*args)
PS D:\大二下\人工智能资料\实验课\lab_7_pytorch和MLP> |
```



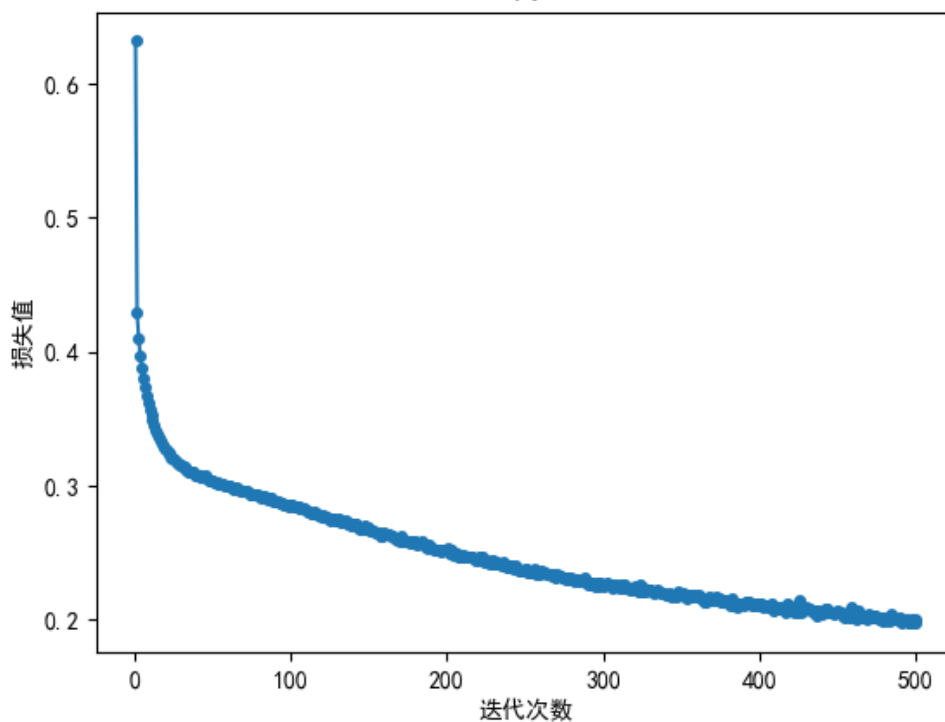
由于我在观察训练过程时发现到 200 轮次损失还在下降，于是我将轮次改为了 500，得到损失（MSE）为 0.22， R^2 为 0.78 训练日志、反向传播损失曲线、训练集上真实值与预测值折线图如下所示。



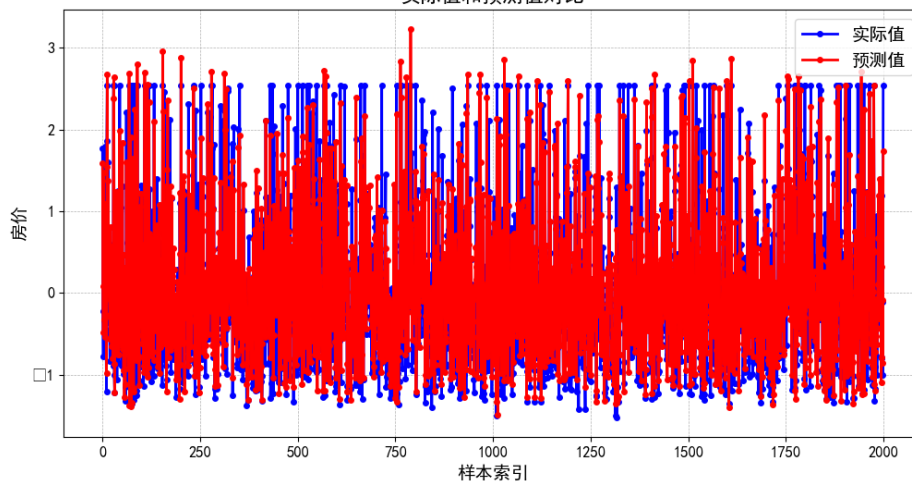
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Epoch 492/500, Loss: 0.19439523208141327
Epoch 493/500, Loss: 0.19516470670700073
Epoch 494/500, Loss: 0.1939192799925804
Epoch 495/500, Loss: 0.19456792390346528
Epoch 496/500, Loss: 0.19574991261959077
Epoch 497/500, Loss: 0.19474121516942977
Epoch 498/500, Loss: 0.19535120004415513
Epoch 499/500, Loss: 0.19682329642772675
Epoch 500/500, Loss: 0.19611903262138367
Test Loss (MSE): 0.22365057468414307
R^2 Score: 0.7820138909533555
D:\anaconda3\envs\unet\lib\tkinter\_init_.py:814: UserWarning: Glyph 8722 (U{MINUS SIGN}) missing from current font.
func(*args)
PS D:\大二下\人工智能资料\实验课\lab 7_pytorch和MLP> █
```

BP迭代曲线



实际值和预测值对比



2. MNIST 数据集分类任务

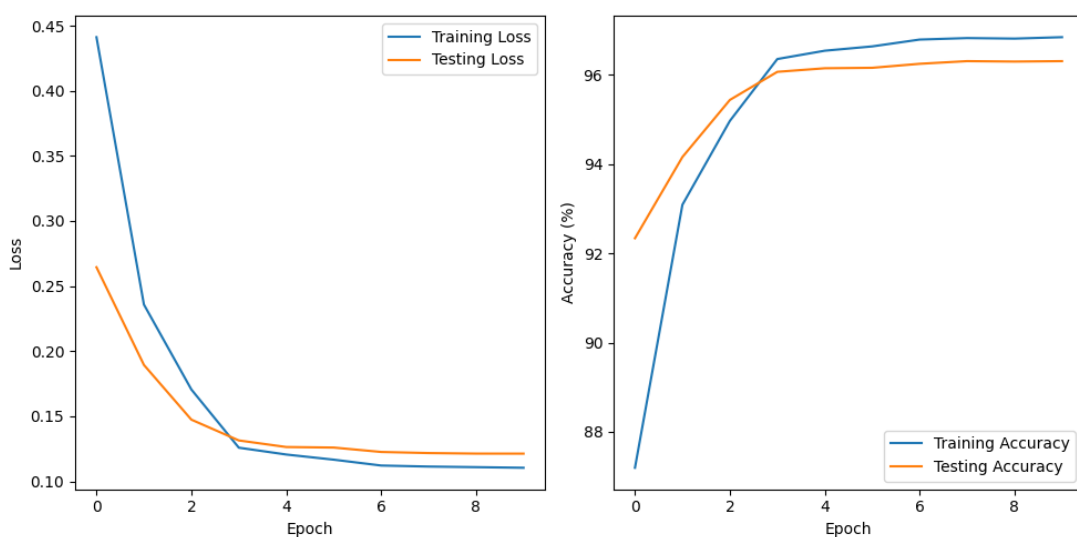
- test_size = 0.2
- random_state = 42
- 输入层神经元数量: 28*28
- 隐藏层 1 神经元数量: 128
- 输出层神经元数量: 10
- batch_size = 128
- epochs = 10
- lr = 0.001
- step_size=3(这里我采用了学习率调度器 Steplr, 每 3 个 epoch

改变一次学习率)

- gamma=0.1 (每次改变学习率时*0.1)
- momentum = 0.9:
- max_norm = 1.0
- nn.CrossEntropyLoss()
- optim.Adam

结果: 使用准确度和损失作为评价标准。经过十个轮次的训练后, 在 test 集上损失为 0.1213, 准确率为 96.31%, 如下图可见训练结果, 及训练过程中每一轮次的训练集和测试集的准确度和 loss 变化曲线。可以看出最开始训练集的损失大于测试集的损失, 之后小于测试集的损失, 而训练集最开始的准确度低于测试集, 之后高于测试集, 证明随着训练轮次的增多, 模型逐渐出现了过拟合的现象。

```
PS D:\大二下\人工智能资料\实验课\lab_7_pytorch和MLP> python -u "d:\大二下\人工智能资料\实验课\lab_7_pytorch和MLP\2.py"
Epoch [1/10], Train Loss: 0.4412, Train Accuracy: 87.19%, Test Loss: 0.2645, Test Accuracy: 92.34%
Epoch [2/10], Train Loss: 0.2358, Train Accuracy: 93.09%, Test Loss: 0.1894, Test Accuracy: 94.16%
Epoch [3/10], Train Loss: 0.1706, Train Accuracy: 94.97%, Test Loss: 0.1474, Test Accuracy: 95.44%
Epoch [4/10], Train Loss: 0.1259, Train Accuracy: 96.36%, Test Loss: 0.1314, Test Accuracy: 96.07%
Epoch [5/10], Train Loss: 0.1206, Train Accuracy: 96.54%, Test Loss: 0.1264, Test Accuracy: 96.15%
Epoch [6/10], Train Loss: 0.1167, Train Accuracy: 96.64%, Test Loss: 0.1260, Test Accuracy: 96.16%
Epoch [7/10], Train Loss: 0.1122, Train Accuracy: 96.79%, Test Loss: 0.1226, Test Accuracy: 96.25%
Epoch [8/10], Train Loss: 0.1114, Train Accuracy: 96.83%, Test Loss: 0.1217, Test Accuracy: 96.31%
Epoch [9/10], Train Loss: 0.1109, Train Accuracy: 96.81%, Test Loss: 0.1213, Test Accuracy: 96.30%
Epoch [10/10], Train Loss: 0.1104, Train Accuracy: 96.85%, Test Loss: 0.1213, Test Accuracy: 96.31%
PS D:\大二下\人工智能资料\实验课\lab_7_pytorch和MLP>
```



3. 总结加整体分析

房价预测任务

在房价预测任务的研究过程中，我深入探究了超参数对模型性能的影响，进行了一系列超参数组合的实验。针对隐藏层神经元数量，我测试了多种不同的配置，像隐藏层 1 分别设置为 50 和 150，隐藏层 2 设置为 30 和 70 等；学习率方面，选取了 0.01 和 0.0001 等具有代表性的值；在优化器的选择上，除了常用的随机梯度下降（SGD），还尝试了 Adagrad、Adadelata 等其他类型的优化器。

实验结果清晰地显示，不同的超参数组合对模型性能有着极为显著的作用。当增加隐藏层神经元数量时，模型的均方误差（MSE）有所降低，同时 R^2 有所提高，这表明模型对房价的预测更加准确。然而，这种改变也伴随着一些问题，训练时间明显变长，而且过拟合的风险大幅增加。这意味着模型在训练数据上表现良好，但在面对未见过的测试数据时，可能无法准确预测，泛化能力有所下降。

学习率的调整同样对模型有着重要影响。当学习率设置得过大时，在训练的初期阶段，模型的收敛速度会明显加快，能够迅速靠近最优解。但由于步长较大，模型很容易错过最优解的精确位置，最终导致 MSE 较大， R^2 较低，使得模型的预测性能大打折扣。

在尝试使用 Adagrad 优化器时，发现它在训练初期确实能让模型快速收敛，损失值下降明显。但随着训练的持续进行，后期学习率会变得过小，导致模型收敛的速度变得极为缓慢，极大地延长了训练时间，最终模型的整体性能不如使用 SGD 优化器的效果。

Mnist 数据集分类任务

针对 Mnist 数据集分类任务，我对隐藏层大小、学习率、优化器等超参数进行了广泛的组合尝试。实验发现，较大的隐藏层大小在一定程度上有助于提升模型的准确率。例如，当把隐藏层大小提升到 256 时，训练集的准确率显著提高。但与此同时，过拟合的问题也随之而来，测试集准确率的提升并不明显，模型在处理新数据时的表现不佳，泛化能力受到影响。

较小的学习率虽然会使训练的收敛速度放缓，但却能够让模型更加精细地寻找最优解。当将隐藏层大小调整为 128、学习率设定为 0.01，并采用学习率调度器 Steplr 后，模型在训练过程中收敛速度没有出现明显变慢的情况，而且准确率也能维持在较好的水平。综合考虑各方面因素，最终决定采用这组超参数，以实现模型训练效率和分类性能之间的平衡。

两个任务最后的训练轮次我都选择的较多，预测房价的轮次选择了 PPT 中的最大轮次，这样在保证模型收敛的前提下不会进行更多轮次浪费计算资源。

思考题

1. 在房价预测任务中，若输入特征存在缺失值，怎么办

处理方法有删除缺失值样本，会减少数据量，可能使模型信息不足、泛化能力下降；用均值、中位数填充，简单但可能引入偏差，影响预测准确性；基于模型预测填充，能利用特征关系，但计算复杂且模型误差会影响填充效果。

2. 在两个实验任务中，都涉及到了损失函数的选择。对于不同的任务，选择合适损失函数的依据是什么？如果改变损失函数，可能会对模型训练和结果产生什么影响？

房价预测选均方误差（MSE）损失函数，因其能直观衡量预测误差。换为平均绝对误差（MAE）损失函数，对异常值更鲁棒，但梯度计算不利，可能延长训练时间。MNIST 分类选交叉熵损失函数，可衡量概率分布差异。换为平方铰链损失，模型训练目标改变，收敛速度和准确率可能变化，影响分类性能。

四、 参考资料

[1]Lab7 参考文档

[2]CSDN: [【深度学习】pytorch 训练 MNIST 数据集 pytorch mnist 数据集-CSDN 博客](#)